



TECHVIZ COMPETITION / APTECH

AuthShield 360

Project field guide and jury preparation

SIGN-IN SEQUENCE

Password > SMS OTP > Email OTP

Role-based school portal with Firebase phone verification and a separate email challenge.

MSG SECURITY SQUAD

Leader: Mustafa Ahmed | Members: Munniba, Aliyan, Mehak, Aisha, Rabia

Mentor: Sir Anas | Competition: TechViz, hosted by Aptech

Prepared 1 October 2026 | Team study and handoff guide

LIVE PORTAL authshield360.vercel.app (recorded, not checked live)

SOURCE github.com/mustafaahmed-sec/AuthShield360

No production credentials, database records or OTP values are included.

Contents

- 1. Project at a glance 3
- 2. Scope and user journeys 4
- 3. How the services fit together 5
- 4. Sign-in: password, SMS, then email 6
- 5. Firebase phone verification 7
- 6. Email OTP 8
- 7. Roles and authorization 9
- 8. Administrator MFA controls 10
- 9. Database schema and relationships 11
- 10. Database and Neon handoff 12
- 11. School workflow 13
- 12. Security controls in source 14
- 13. Rate limits and challenge timing 15
- 14. Local verification record 16
- 15. Hosting and release path 17
- 16. What each platform does 18
- 17. Folder structure and handoff contents 19
- 18. Local setup for a new maintainer 20
- 19. Privacy, secrets and ownership 21
- 20. Current boundaries and remaining evidence 22
- 21. Architecture choices and trade-offs 23
- 22. Jury questions: system and data 24
- 23. Jury questions: authentication and security 25
- 24. Future work 26
- 25. Summary and references 27

1. Project at a glance

AuthShield 360 is a Django school portal for Students, Teachers and Administrators. It handles access requests, approvals, role dashboards, courses, enrollment, assignments, results, attendance, announcements and security events. When role policy and provider configuration require both factors, sign-in follows password, Firebase SMS OTP, then separate email OTP.

Project item	Handoff facts
Competition	TechViz, hosted by Aptech. Team: MSG Security Squad.
Team	Mustafa Ahmed (leader); Munniba, Aliyan, Mehak, Aisha and Rabia; mentor Sir Anas.
Source	github.com/mustafaahmed-sec/AuthShield360
Recorded portal	https://authshield360.vercel.app . Current availability was not checked.
Technology	Django/Python, PostgreSQL migrations, Firebase Phone Auth, mail provider, GitHub Actions and Vercel.
Credits	Team reports USD 300 in Firebase credits. Current balance and eligible services were not checked.

Package: source, schema handoff, one PDF guide and one editable presentation. These files do not transfer cloud accounts or prove a live production release.

Purpose

Schools need different users to see different records. The portal applies role permissions on the server and can require phone and email verification after a password.

2. Scope and user journeys

Student and Teacher registrations start as requests. An Administrator reviews a request before approved access. Request status, an approved account and a student record are separate states.

Role	Typical work	Boundary
Student	View own permitted profile, course, assignment, attendance and result information.	No access to other students private records or administrator controls.
Teacher	Work with assigned courses and class records, including attendance.	No request approval or global MFA controls.
Administrator	Review requests, manage accounts, inspect security events and set MFA policy.	Sensitive operations require server-side authorization.

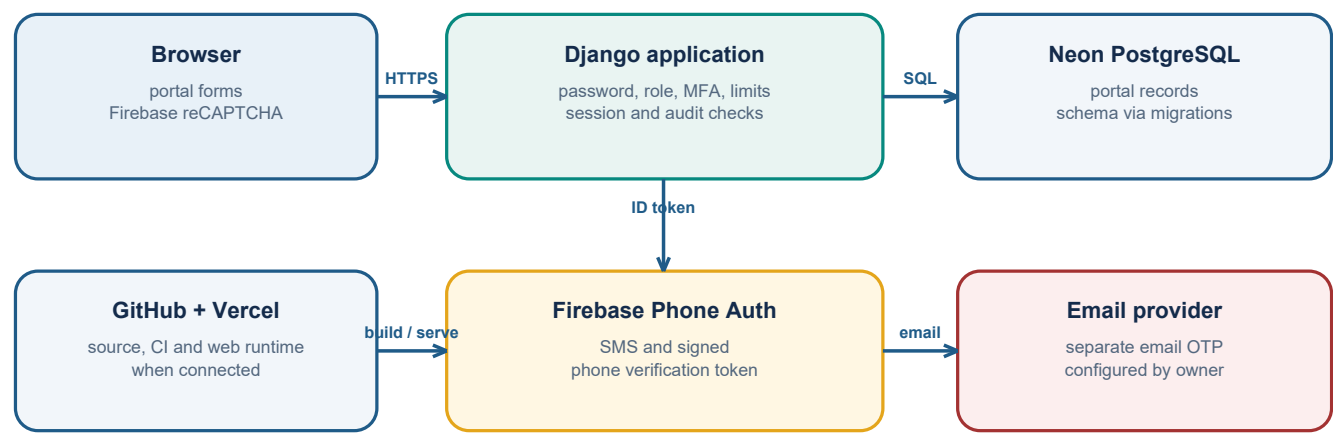
Included workflows

Account request and review, password sign-in, phone and email challenges, request status, password recovery, account administration, school records, audit events and administrator MFA settings.

This is a web portal. No Android APK or final demo video is included.

3. How the services fit together

Each service has a separate responsibility. Django is the application decision point; PostgreSQL stores application records; Firebase runs phone verification; the configured email provider sends email challenges. GitHub holds source and checks. Vercel runs the application when the intended repository and project are connected.

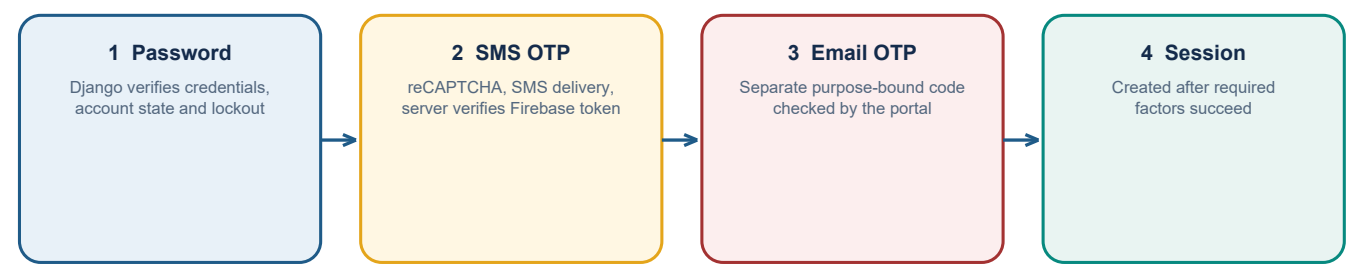


Service	Role	Boundary
Django	Requests, password and role checks, challenge state, limits and pages.	Local code does not prove production configuration.
PostgreSQL / Neon	Account, school, policy, quota and audit records.	Migrations are included; production rows and Neon access are not.
Firebase Phone Auth	Web reCAPTCHA, phone verification and SMS delivery.	Cannot guarantee carrier delivery or account quota.
Email provider	Delivers separate email OTP and account messages.	Credentials and inbox delivery are not verified.
GitHub and Vercel	Version history, checks, build and web deployment.	Latest workflow and deployment need provider evidence.
Keycloak, optional	OIDC sign-in or authenticator-app TOTP if operated and configured.	Source folder alone is not a live identity service.

Cloud accounts remain with their owners. Copying files does not transfer Firebase, Neon, GitHub or Vercel access.

4. Sign-in: password, SMS, then email

When policy requires both available factors, the application follows this order. Each factor has a different verification result.



Stage	What happens	Pass evidence
Password	Django checks credentials, approval, role and lock state.	Server-side password and account-state check.
SMS OTP	The browser completes reCAPTCHA and requests Firebase Phone Auth.	Server verifies the signed Firebase ID token and expected account phone.
Email OTP	Django sends a separate email challenge if policy requires it.	Code matches its stored hash within its purpose, expiry and attempt rules.
Session	Django completes sign-in after required checks.	Authenticated session with role checks on later requests.

Typing digits, browser autofill or seeing a success banner is not enough by itself. The backend must verify the challenge result.

If email verification fails after SMS passed, only the email challenge should need retry or restart, subject to portal state. SMS and email codes are not interchangeable.

5. Firebase phone verification

Firebase Phone Authentication for the web runs a reCAPTCHA security check before the browser requests a phone code. After a valid code, Firebase returns a signed ID token. Django verifies the token before trusting the phone result.

What the user sees

- A masked registered phone number.
- A reCAPTCHA control before the SMS request.
- A dedicated code input that can work with browser suggestions.
- A recoverable error if Firebase, the carrier or browser rejects the attempt.

Why delivery can vary

One successful SMS proves that one request was delivered at that time. Later requests can be rejected or delayed because of carrier or region rules, Firebase abuse controls or quota, billing, project settings, reCAPTCHA/App Check configuration, domain authorization, phone formatting or network conditions. This review did not request a real code.

Owner checks

- Confirm the production domain is authorized in the correct Firebase project.
- Confirm Phone sign-in, billing and SMS settings are active.
- Check reCAPTCHA/App Check, country policy, usage and recent error metrics.
- Test with an authorized test account and record provider evidence.

Official reference: [Firebase Phone Authentication for web apps](#).

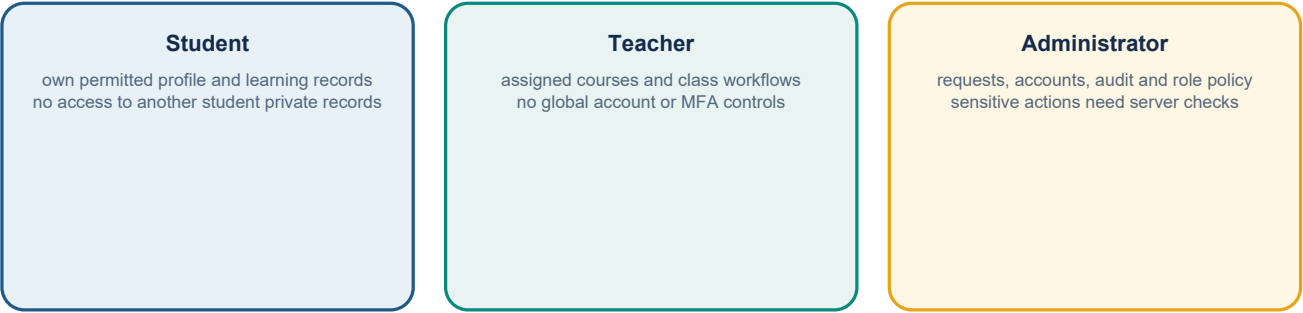
6. Email OTP

Email OTP is a separate portal challenge after SMS in the requested flow. Firebase does not send this code. The application should identify whether the message concerns sign-in, account verification or password reset, and show the role or account name when available.

Concern	Expected behavior
Purpose	Email states why a code was sent.
Input	Dedicated field supports normal copy and paste.
Expiry	Expired codes are rejected.
Attempts	Repeated guesses are bounded by server limits.
Storage	Code is hashed for verification and should not appear in logs.
Delivery	The page reports success only after the mail backend accepts the send.

Production mail credentials, sender reputation, inbox placement and delivery were not checked in this source-only review.

7. Roles and authorization



Django enforces the role boundary on protected requests.

Django must check access for every protected request. The browser may hide actions to improve usability, but hiding a button does not protect an endpoint.

Action	Student	Teacher	Administrator
View permitted own records	Yes	Yes	Yes
Manage assigned class workflows	No	Yes	As allowed
Approve access requests	No	No	Yes
Set role MFA policy	No	No	Yes
Review security events	Limited	Limited	Administrative

A screenshot alone does not prove authorization. Show the server-side permission rule for the example.

8. Administrator MFA controls

The administrator dashboard stores separate portal MFA policies for Students, Teachers and Administrators. An administrator can choose enabled factors and whether both configured factors are required, then confirm the save. The policy applies to future sign-ins and can invalidate pending OTP sign-ins for the affected role. Existing authenticated sessions do not automatically prove fresh MFA.

Control	Effect	Operational consequence
Enable MFA for a role	Future sign-ins use the enabled and available factors.	An unavailable required provider may block the user.
Choose SMS and email	The portal applies both channels in order.	SMS first, then a separate email code.
Turn a role policy off	Removes the portal OTP step on later sign-ins.	This weakens protection and should be limited and logged.
Keycloak TOTP setting	Separate optional control, dependent on Keycloak Admin API and flow configuration.	Production operation was not confirmed.

A portal switch does not configure Firebase or an external Keycloak realm, and it does not automatically revoke existing sessions.

9. Database schema and relationships

This page summarizes the Django application schema. Exact fields, constraints and nullability live in `accounts/models.py`, `school/models.py` and the tracked migrations. The production database itself was not read or exported.

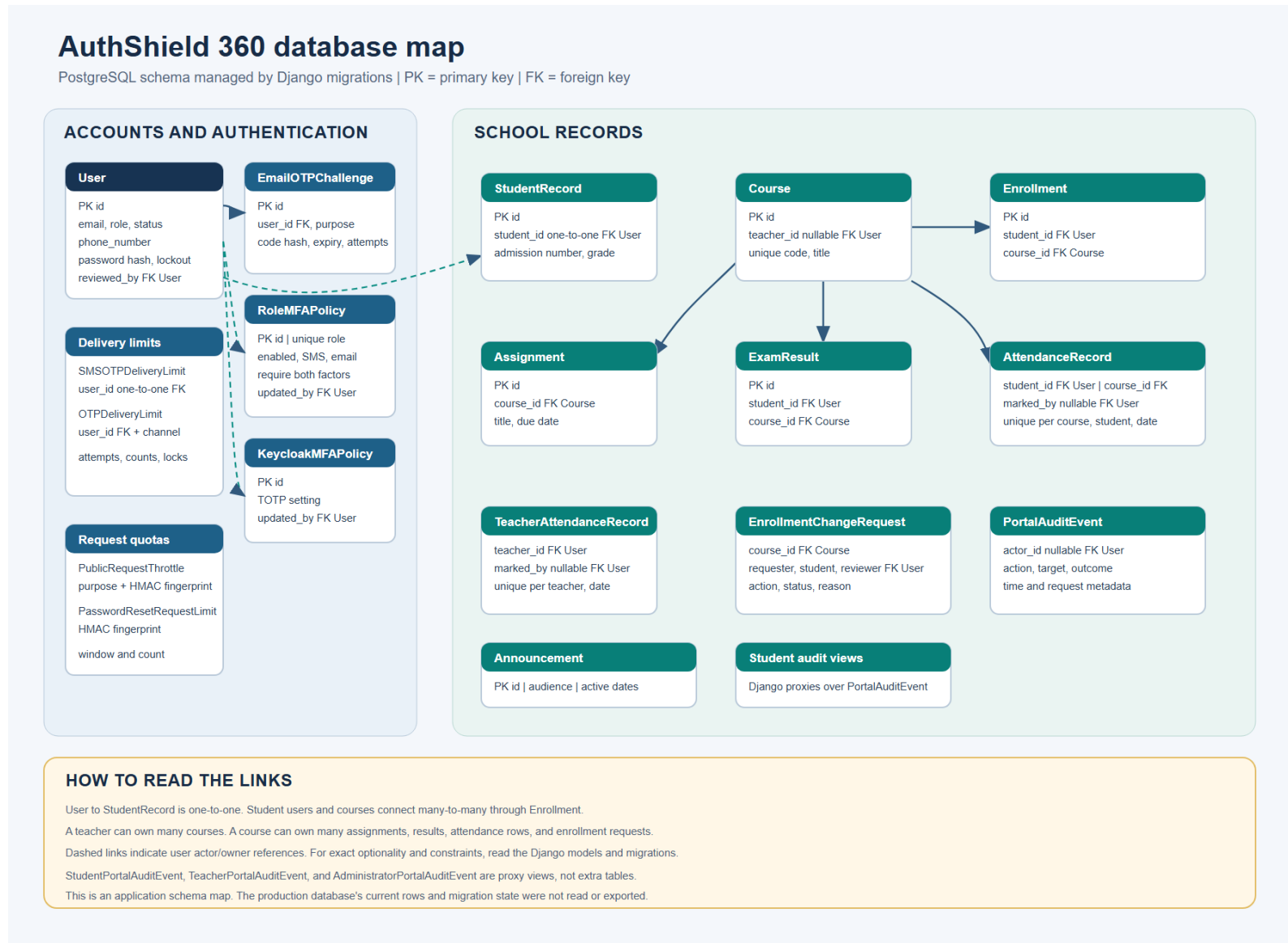


Figure: application schema map. Proxy audit views reuse the PortalAuditEvent table; they are not separate physical tables. Read model definitions and migrations for exact constraints.

10. Database and Neon handoff

PostgreSQL stores structured portal records. Neon is the hosted PostgreSQL provider documented for this project. Django migrations make schema changes reproducible; they do not contain production records.

Database terms

- Table: records of one type, such as users or courses.
- Primary key: the unique identifier for a row.
- Foreign key: a database link from one row to another.
- Migration: a versioned schema change applied by Django.
- Backup: a separate copy of database records restored through an authorized provider account.

Included

- Schema migrations under `accounts/migrations/` and `school/migrations/`.
- `Database/ERD.svg` and `Database/DATABASE_HANDOFF.txt`.
- Instructions for a local database.

Not included

- Production SQL dump, real user data, Neon password or provider token.
- Neon account ownership. An authorized owner must grant access or provide an approved backup.

Official reference: [Neon workflow primer](#).

11. School workflow

A student or teacher begins with an access request. An Administrator reviews it. Approved accounts enter role-specific portal pages. School records link courses to students and connect assignments, results and attendance to course or user records.

Workflow	Checks and records	Outcome
Student request	Form, pending state, administrator review.	Approve or reject access.
Teacher request	Form, pending state, administrator review.	Enable role access only after review.
Course work	Teacher assignment, roster and class records.	Students see only authorized course information.
Request status	Submitted details match a request record.	Shows request state; does not create an approved account.
Password recovery	Purpose-specific email challenge and send limits.	A verified user can reset a password under configured rules.

Why states matter

A request, approved portal account and student school record are related but separate. This distinction prevents a pending request from being described as an active student account.

12. Security controls in source

Identity	password, role, approval
Challenge	reCAPTCHA, SMS, email, expiry
Abuse controls	quotas, cooldowns, logout
Authorization	CSRF, server-side role checks
Evidence	audit, tests, changed-code review

Control	Purpose	Evidence
Password and account checks	Protect sign-in and reject disallowed account states.	Django auth, role and approval checks, logout tests.
CSRF and secure settings	Protect browser-originated changes and production cookies/transport.	Settings and framework checks.
OTP rules	Limit lifetime, attempts and resends; hash email challenges.	Portal code and automated tests.
Quota rows	Apply account/source limits across application processes.	Database-backed throttles and regression tests.
Audit events	Record selected security and administrator outcomes.	Audit helpers and tests.
Preview DB guard	Prevent preview from selecting production DB through URL encoding ambiguity.	Canonicalization code and regression tests.

These controls reduce specific risks. No application can honestly be called unhackable, and an automated scan cannot certify the live service against every attack.

13. Rate limits and challenge timing

The table shows the current documented project defaults. Deployment settings can override some values. Firebase applies its own provider-side policy.

Control	Documented default	Purpose
Sign-in OTP lifetime	5 minutes for Firebase SMS; email has its configured TTL.	Limit code reuse time.
Resend wait	5 minutes between sign-in sends when applicable.	Reduce repeated SMS cost and abuse.
Send limit	3 sends per account/channel in configured window, then a 30-minute lock.	Bound delivery attempts.
Code guesses	Up to 5 challenge attempts.	Slow automated guessing.
Password reset sends	3 per account per hour plus an IP request limit.	Reduce email abuse and account enumeration.
Role denial audit	Repeated events throttled using privacy-preserving quota state.	Keep audit records useful without readable raw addresses.

Source defaults do not prove the production environment uses the same values. Check deployment settings before promising exact live limits.

These are project choices, not a claim that OWASP mandates one fixed value for every portal.

14. Local verification record

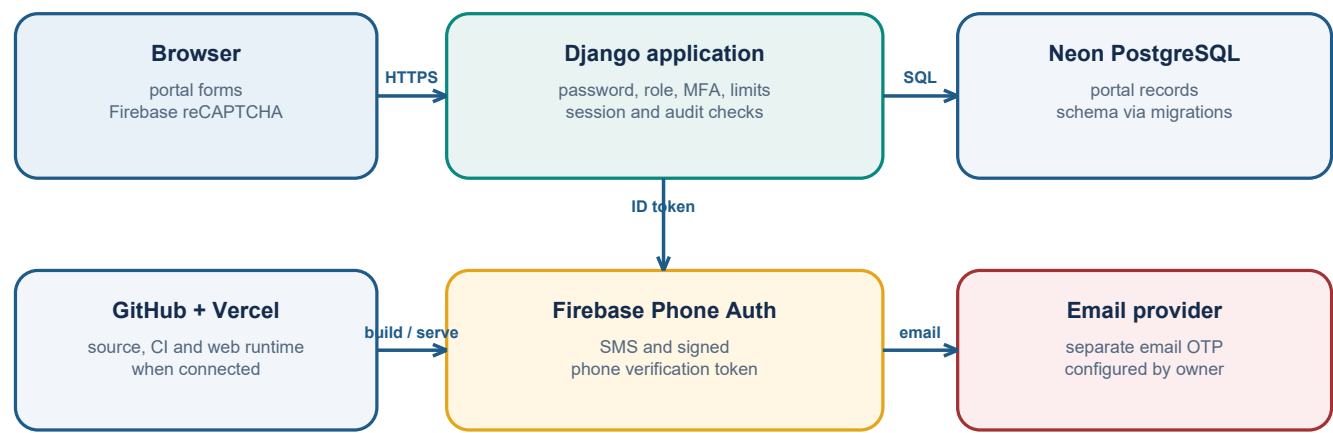
The team recorded local checks in isolated settings. The test configuration uses in-memory SQLite and in-memory email, so those checks do not call the production database or send real messages.

Check	Recorded result	Boundary
Django tests	145 tests passed.	Local test settings only.
Ruff	Passed.	Lint and style checks.
Django system check	No issues reported.	Configuration checks under test settings.
Migration drift	No pending model migrations reported.	Local model and migration consistency.
Codex Security diff review	Completed; 13 changed security-relevant files reviewed; zero reportable findings.	Changed-code review, not penetration testing.
Real SMS or email	Not performed during this review.	No OTP requested or sent.
Production services	Not checked.	No live login, database, Firebase, Keycloak or Vercel access.

The reviewed source changes covered MFA policy, OTP input validation, password-reset quotas, audit throttling and preview database URL handling. A source review cannot prove carrier delivery or production behavior.

15. Hosting and release path

The intended path stores source in GitHub and connects a branch to a Vercel project. Vercel builds and serves the application; protected environment settings connect it to the database and identity providers.



Owner evidence	What to confirm
GitHub	Latest main commit and workflow result.
Vercel	Production deployment commit, build log and domain.
Neon	Production branch, applied migration state, backup and restore test.
Firebase	Project, authorized domain, billing, reCAPTCHA/App Check, SMS policy and usage.
Email	Provider credentials, sender identity, delivery evidence and limits.
Keycloak	Active realm, service account, authentication flow and authorized test.

References: [Vercel Functions](#), [Vercel deployments](#), [GitHub Actions](#).

16. What each platform does

Platform	Beginner explanation	Project role
Django	Python framework for requests, forms, database models and account rules.	Portal backend and pages.
PostgreSQL	Relational database with keys, constraints and transactions.	Stores account, school, policy, quota and audit data.
Neon	Hosted PostgreSQL service with database branches.	Documented hosted database.
Firebase	Google service suite; Phone Auth runs web phone verification.	reCAPTCHA, SMS request and signed phone result.
Vercel	Build and hosting platform for web apps and functions.	Hosts application when connected and configured.
GitHub	Git source hosting and workflow automation.	Version history, code review and CI.
Keycloak	Identity and access management supporting OIDC and authenticator flows.	Optional OIDC/TOTP; production setup is not confirmed.

References: [Keycloak administration](#), [Firebase Phone Auth](#).

17. Folder structure and handoff contents

Keep one project root on the USB and one matching working copy on C:. Source files stay at the root; do not create a second nested AuthShield360 source tree.

Path	Purpose	Handoff note
accounts/	Users, MFA, OTP, login and request workflows.	Includes migrations and tests.
school/	Courses, records, permissions and audit.	Includes migrations and tests.
config/	Django settings, routes and runtime entry points.	Use private environment configuration.
templates/ and static/	Portal pages, styles, browser scripts and referenced artwork.	Keep used logo and favicon assets.
Database/	ER diagram and setup handoff.	Schema, not production data.
Documentation/	One PDF guide and one editable PowerPoint.	Final versions only.
keycloak-vercel/	Optional Keycloak service source.	Not proof of a live identity server.
README.md and LIVE_LINK.txt	Quick start and recorded project URLs.	URL must be checked separately.

Excluded items

- No video folder because no final video was provided.
- No APK folder because the deliverable is a web portal.
- No production database export, cloud secret, email app password or OTP.
- The access workbook and portal credential text remain local on C: and USB. Git ignores them and Vercel packaging excludes them. Anyone who receives the USB may still read them.

18. Local setup for a new maintainer

Use a disposable local database and fictional demonstration records. Never point demo seed or cleanup commands at production.

Step	Action
1	Install Python 3.12 and PostgreSQL. Create a restricted local database user.
2	Copy <code>.env.example</code> to a private <code>.env</code> . Set a new local Django secret and local database URL.
3	Install dependencies with <code>python -m pip install -r requirements-dev.txt</code> .
4	Apply schema with <code>python manage.py migrate</code> .
5	Create a local administrator with <code>python manage.py createsuperuser</code> if needed.
6	Use fictional demo records only in a disposable local database.
7	Run with <code>python manage.py runserver 127.0.0.1:8000</code> .
8	Run tests with isolated settings; run Ruff and Django checks as shown in README.

Do not paste production secrets into `.env.example`, GitHub, this guide, the presentation or screenshots.

19. Privacy, secrets and ownership

A technical handoff includes code and instructions. Operational ownership also needs provider access, recovery contacts, billing responsibility, database backups and environment settings.

Item	Included?	Owner action
Django/Firebase public config	Only values intentionally exposed to the web client.	Restrict Firebase API key and authorized domains.
Django secret and provider passwords	Not included in this guide or committed source.	Set fresh values in the receiving owner account.
Access workbook and portal credential text	Local on C: and USB; ignored by Git and excluded from Vercel.	Encrypt and share only with an authorized administrator.
Neon production rows	Not included.	Owner grants access or makes an approved encrypted export.
Provider accounts	Not transferred by copying files.	Invite the next admin with each provider access-control system.

If an untrusted person receives the USB, assume they can inspect local files on it. Remove private access files before handing the device to anyone who is not authorized.

20. Current boundaries and remaining evidence

Local code and test results are useful evidence. These provider checks need the service owner or a live authorized test.

Boundary	Why it matters	Evidence to collect
Vercel release	Local source may differ from production.	Production commit, successful build and domain check.
Neon database	Migration files do not prove production migrations or backups.	Owner checks branch, migration state and restore.
Firebase delivery	Past delivery does not prove current carrier routing or quota.	Authorized SMS test and provider usage evidence.
Email delivery	In-memory tests do not prove inbox delivery.	Test inbox and provider logs.
Keycloak MFA	Optional source does not prove realm, Admin API or flow works.	Active realm and authorized TOTP/OIDC test.
SRS traceability	No separate SRS PDF is in this handoff directory for line-by-line mapping.	Compare the approved SRS against final behavior.

Describe tested local behavior separately from live production evidence. Do not claim an external service is currently working without a current check.

21. Architecture choices and trade-offs

Choice	Reason	Trade-off
Django server rules	One backend owns request, permission, challenge and school logic.	Keep dependencies patched and deployment settings correct.
PostgreSQL	Relational school records fit keys, constraints and transactions.	Cloud access, migration discipline and backups are separate duties.
Firebase Phone Auth	Managed phone verification and SMS delivery.	Carrier, country, quota, billing and abuse policy affect delivery.
Email as a second factor	Adds a separate challenge after SMS.	Email account security and provider deliverability matter.
Vercel hosting	Builds and serves a web app from a connected project.	Usage caps and project settings can affect releases.
Optional Keycloak	Can centralize OIDC identity and authenticator policy.	Needs a reachable server, secure Admin API and tested flow.

These are project choices with trade-offs, not claims that one provider is universally best.

22. Jury questions: system and data

Why Django?

Django supplies URL routing, forms, authentication tools, database models, migrations and server-side permission checks in Python. The application keeps important decisions on the backend.

Why PostgreSQL and Neon?

PostgreSQL stores linked school data with keys and transactions. Neon hosts PostgreSQL. Django migrations describe schema changes; they do not copy production rows or cloud ownership.

How do roles protect records?

Django checks the authenticated account role and status before returning a protected page or accepting a change. Hiding a menu item is not an access control.

How are Students and Teachers added?

They submit access requests. An Administrator reviews and approves or rejects them. A request is not the same as an active account.

Is production data on the USB?

No. The handoff has schema migrations and a diagram. An approved production backup belongs in an encrypted owner-controlled process.

23. Jury questions: authentication and security

Why SMS before email?

The project brief specifies password, SMS OTP, then email OTP. The application checks the challenges separately before opening the portal session.

Does Keycloak send Firebase SMS?

No. Firebase Phone Auth handles that SMS flow. Keycloak can provide OIDC sign-in or authenticator-app TOTP if its service is active and configured.

What if SMS does not arrive?

Firebase or a carrier can reject or delay delivery. The portal can enforce a retry wait but cannot force delivery. The owner should check Firebase settings and usage metrics.

Can repeated requests create cost?

The portal applies its own quotas and cooldowns. Firebase also applies provider-side limits. Owners should monitor both and set conservative project values.

Is the portal unhackable?

No responsible team should promise that. Source controls and local checks reduce risk, but this review is not a live penetration test or a guarantee against every attack.

What does zero findings mean?

No reportable issue survived review of the specified changed-code scope. This does not certify every dependency, file, cloud setting or production endpoint.

24. Future work

These improvements need owner access to connected services or additional authorized testing.

- Add a browser test with Firebase fictional numbers to exercise the challenge page without sending real SMS.
- Record the latest Vercel production commit, build result and domain check in release notes.
- Verify Neon migrations and restore a backup to a separate branch.
- Collect Firebase delivery and usage evidence for supported countries and carriers.
- Collect email delivery, bounce, sender and rate-limit evidence.
- Finish Keycloak Admin API and authentication-flow setup only if the project will actively use OIDC or TOTP.
- Add accessibility review, dependency monitoring and an incident recovery exercise.
- Consider passkeys or authenticator-app MFA as future options with accessible recovery.

For the jury, distinguish local test evidence from live provider evidence.

25. Summary and references

AuthShield 360 is a role-based Django school portal backed by PostgreSQL. Firebase handles phone verification and SMS; a separate mail provider delivers email OTP. GitHub stores tracked source and checks. Vercel hosts the application when its connection is configured. Keycloak remains optional until its live service and flow are verified.

Official references

[Django security](#) | [Django authentication](#) | [Django database models](#)

[Firebase web phone authentication](#) | [Neon workflow primer](#)

[Vercel Functions](#) | [Vercel deployments](#) | [GitHub Actions](#) | [Keycloak administration](#)

One-sentence presentation summary: AuthShield 360 checks a password and, when required by role policy, verifies a phone through Firebase SMS and a separate email code before opening protected portal access.